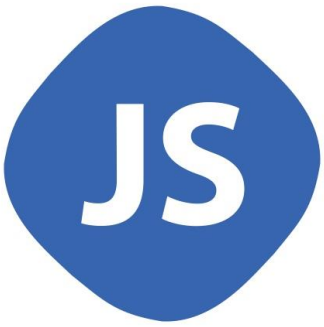


Диалоговые окна

- Введение
- Метод alert ()
- Метод confirm ()
- Метод prompt ()
- Итог

Введение

В этом уроке разберемся, как выводить **диалоговые окна**, позволяющие организовать **взаимодействие с пользователем**.

Если вы пишете программу на JavaScript, то можете воспользоваться тремя **стандартными** методами для ввода и вывода данных:

- **alert**();
- **confirm**();
- **prompt**().

Каждое из этих диалоговых окон является **синхронным** и **модальным**, то есть выполнение кода приостанавливается на время показа такого окна и возобновляется после его закрытия.

> ***К сведению***. Окна, обладающие свойством блокировать работу пользователя с родительским приложением до тех пор, пока пользователь это окно не закроет называются **модальными**. Модальные окна часто используются для привлечения внимания пользователя к важному событию или критической ситуации.





Метод alert ()

Метод **alert**() позволяет выводить диалоговое окно с заданным сообщением и кнопкой OK. Синтаксис метода имеет следующий вид:

```
alert ('Сообщение');
```

Методу alert() передается только один аргумент — отображаемое сообщение. В простейшем случае текст сообщения, заключенный в двойные или одинарные кавычки, вводится внутри круглых скобок. В более сложном случае текст сообщения может быть представлен в виде предварительно созданной переменной JS. Сообщение может быть как строкового, так и числового типа.

Диалоговые окна оповещений обычно применяют, если нужно уведомить пользователя о каких-либо его действиях, или вывести сообщение об ошибке.

Диалоговое окно, выведенное на экран методом alert(), можно убрать, кликнув на кнопке **OK**. До тех пор, пока вы не сделаете этого, переход к ранее открытым окнам невозможен.

example_1. Метод alert

```
<script>
      alert("Добро пожаловать в мир JavaScript!");
</script>
```

Чтобы отобразить многострочное сообщение, строки можно разделять управляющим символом переноса строки – **\n**.

example_2. Многострочная строка в методе alert()

```
<script>
      // инициализируем строку
      // строка содержит управляющий символ - перенос строки
      let msg = 'Привет! Мы будем изучать JS,\nКто с нами,
      присоединяйся!';
```



```
    // отдаем строку в качестве аргумента
    alert(msg);
</script>
```

Формируемые строки могут быть достаточно сложными и содержать не только управляющий символ переноса строки, но и переменные.

example_3. Вставка переменных в строку вывода метода alert()

```
<script>
    // формируемые строки могут быть достаточно сложными
    let user = "Алиса";
    let msg = 'Привет ' + user + '! Мы будем изучать:\n ' +
    '- PHP\n' +
    '- JavaScript\n' +
    '- Python\n' +
    'Желаю удачи!';
    // вывод строки
    alert(msg);
</script>
```

Окно предупреждения генерируется самим браузером, поэтому внешний вид окна в различных браузерах может несколько различаться.

Программисты на JS часто наряду с **выводом в консоль** используют вставку в программный код метода **alert**() с целью **диагностики возникающих проблем** (и их анализа).

Метод confirm ()

Метод **confirm**() позволяет вывести диалоговое окно с сообщением и двумя кнопками — **OK** и **Отмена** (или **Да** / **Нет** - в зависимости от браузера),

благодаря чему пользователь может подтвердить или отменить некоторое действие.

В отличие от метода **alert** этот метод возвращает **логическую величину**, значение которой зависит от того, на какой из двух кнопок кликнул пользователь.

Чтобы можно было определить, какую кнопку выбрал пользователь, метод **confirm** возвращает **true**, если был клик на кнопке **OK**, и **false**, если клик на кнопке **Отмена** или системной кнопке **закрытия окна** — X.

> ***Еще раз***. Кнопка OK - **true**. Кнопка Отмена или закрытие окна - **false**.

Синтаксис применения метода confirm() имеет следующий вид:

```
var result = confirm('Сообщение');
```

где:

- **result** — это логическое значение **true** / **false**, указывающее, было ли выбрано **OK** или **Отмена**. Возвращаемое значение можно использовать в дальнейших выражениях.

example_4. Метод confirm

```
<script>
    // нажимаем OK
    let ok = confirm("На ваш счет будет переведен миллион долларов. Вы согласны?");
    console.log(ok);  // true
    // нажимаем Отмена
    let undo = confirm("С вашего счета будут списаны все денежные средства. Ok?");
    console.log(undo);  // false
</script>
```



Пример использования **результата** выбора представлен в следующем примере. Метод confirm() выводит сообщение и ожидает, пока пользователь подтвердит или отменит некоторую операцию, кликнув на одной из кнопок.

example_5. Диалоговое окно принятия решения

```
<script>
      // анализируем реакцию пользователя на диалоговое окно
      if (confirm("Молочный коктейль за 5 долларов заказывали?")) {
            alert ("Получите бургер Биг Кахуна в подарок!");
      } else {
            alert ("Он не стоит пяти баксов...");
      };
      // условные конструкции if-else будем изучать позже
</script>
```

Таким образом выглядит типичный код вызова диалогового окна confirm.

В этом примере оператор if вычисляет условие, и если результат – **true** (пользователь кликает на кнопке **OK**), то выполняет команду и выводит сообщение: "Получите бургер Биг Кахуна в подарок!".

Блок else выполняется, если пользователь выбирает кнопку **Отмена**.

Условные конструкции **if-else** будем изучать чуть позже.

Метод prompt ()

Метод **prompt**() выводит на экран диалоговое окно, которое запрашивает у пользователя информацию.

Вместе с кнопками **OK** и **Отмена** оно содержит текстовое поле для ввода данных. В отличие от методов alert() и confirm() данный метод принимает два

параметра: **сообщение** и **значение**, которое должно появиться в текстовом поле ввода данных **по умолчанию**.

Синтаксис применения метода prompt имеет следующий вид:

```
var result = prompt("Сообщение", "Значение по умолчанию");
```

- **result** — строка, содержащая текст, введенный пользователем, или значение **null**.
- **Сообщение** — строка текста для отображения пользователю. Этот параметр является **необязательным** и может быть опущен, если в окне подсказки ничего не должно отображаться.
- **Значение по умолчанию** — строка, которая отображается по умолчанию в поле ввода. Второй аргумент можно оставить пустым или записать так - "".

Если пользователь кликает на кнопке **OK**, метод prompt() возвращает значение, введенное в текстовое поле, а если выбирается кнопка **Отмена** или окно закрывается иным образом, то возвращается **null**.

Вот пример.

example_6. Метод prompt

```
<script>
    // нажмите OK
    let str1 = prompt("Нажмите OK.\nОтклик в консоле - пустая
    строка.");
    console.log(str1);
    // нажмите OK
    let str2 = prompt("Нажмите OK.\nПроверьте отклик в консоле.",
    "Привет, вы нажали OK");
    console.log(str2);
    // нажмите Отмена
    let undo = prompt("Нажмите Отмена.\nПроверьте отклик в консоле.",
    "Вы нажали Отмена");
```

```
        console.log(undo);
</script>
```

Следующий листинг представляет вариант взаимодействия диалоговых окон с пользователем.

example_7. Тестируем взаимодействие диалоговых окон

```
<script>
      // переменная name получит значение, введенное пользователем
      // или значение по умолчанию
      var name = prompt("Здравствуйте, представьтесь, пожалуйста",
      "noName");
      // диалоговое окно принятия решения
      if (confirm("Вы хотите просмотреть скрытый раздел?")) {
            // если пользователь выбрал Ok
            alert("Рады видеть вас, уважаемый(ая) "  + name + "!");
            document.write("<h2>Раздел персональных данных пользователя
            " + name + "</h2>");
      } else {
            // если пользователь выбрал Отмена
            document.write("<h2>Ждем вас еще!</h2>");
      }
</script>
```

Итог

Методы **alert**(), **confirm**() и **prompt**() являются блокирующими, т. е. они не возвращают управление документом, пока пользователь не отреагирует на запрос диалогового окна. Это значит, что, когда выводится одно из таких окон, программный код прекращает исполнение, а текущий документ прекращает

загружаться на время показа диалогового окна и возобновляется после его закрытия.

- **alert**() выводит сообщение и ждёт, пока пользователь нажмёт **OK**;
- **confirm**() выводит сообщение и ждёт, пока пользователь нажмёт **OK** или **Отмена** и возвращает соответственно **true** или **false**;
- **prompt**() выводит сообщение и ждёт, пока пользователь введёт текст, а затем возвращает **введённое значение** или **null**, если ввод отменён (**Отмена** или **Esc**).

Такие системные диалоговые окна легко использовать, они могут быть полезны, если нужно вывести для пользователя некоторую информацию и запросить подтверждение решения, не требуя загрузки кода HTML, CSS или JavaScript.

Однако из эстетических соображений на практике их применяют редко. На веб-страницах они почти не встречаются. Текст, отображаемый в диалоговых окнах, которые выводятся перечисленными методами – это обычный неформатированный текст. Его можно форматировать только пробелами, переводами строк и различными знаками пунктуации.

> ***Примечание***. Современные разработчики веб-приложений вместо применения стандартных возможностей диалоговых окон предпочитают создавать свои, используя множественные CSS настройки, либо сторонние CSS-библиотеки. Плюс такого подхода – встраивание диалогов в общий дизайн сайта, минус – объем кода и требование к знаниям дополнительного материала.

Чтобы не быть голословным, приведу пример всплывающего модального окна с использованием библиотеки Bootstrap. Кто незнаком с библиотекой – вот вам еще одно направление для изучения. HTML – CSS – JavaScript – Bootstrap – вот в чем сила, брат ;)

Ну, а далее везде (PHP – MySQL – Laravel – WordPress – ....)

Удачи.

example_8. Всплывающее окно средствами Bootstrap

```
<!--
Ввиду объемного кода полный листинг приводить не буду. Открывайте
файл, изучайте.
Важно! Для тестирования кода ваш компьютер должен быть подключен к
глобальной сети Интернет.
-->
```

Примечание. HTML, CSS, Bootstrap – в нашем курсе "Основы HTML и CSS. С нуля до первого сайта".

Самостоятельные работы:

- Формирование строк для вывода сообщений диалогового окна;
- Динамическое присвоение значений свойствам объекта;
- Вставка данных объекта в форматированные строки;
- Создание и динамическое заполнение свойств объекта с типом данных – массив;
- Работа с диалоговыми окнами JS.